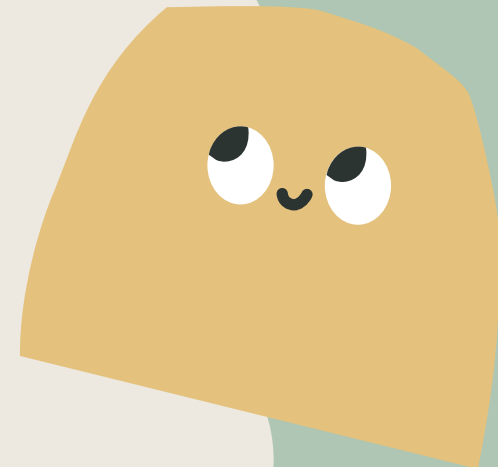
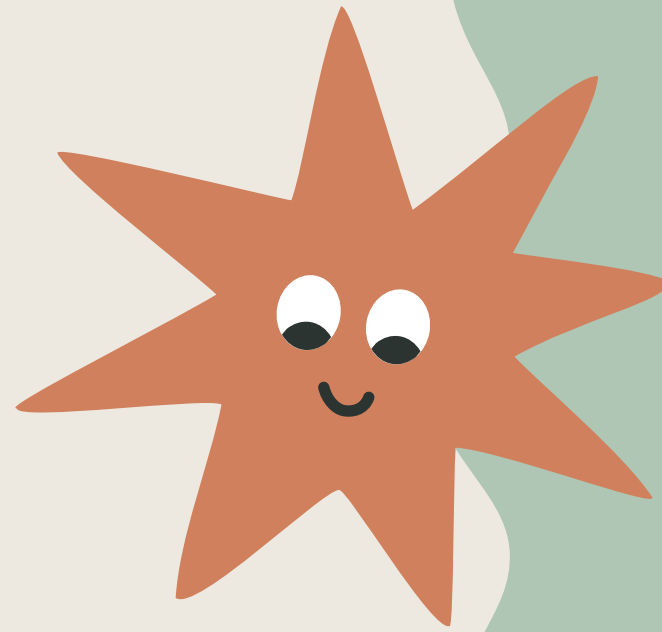


Algorithms, Problem Solving & Abstract Data Types (ADT)

Lecture 2



Learning Outcomes (Lecture 2)

➤ At the end of this lecture, students will be able to:

- Explain what an algorithm is.
- Identify the characteristics of a good algorithm.
- Differentiate between an algorithm and a program.
- Draw basic flowcharts.
- Explain the concept of an Abstract Data Type (ADT).
- Differentiate between an ADT and a Data Structure.
- Understand how ADTs are implemented using different data structures.

Why Do We Need Algorithms?

- Suppose we want to make a cup of tea
- Can we randomly start doing things?
- No... We need a sequence
 1. Boil water
 2. Add tea
 3. Add sugar
 4. Add milk
 5. Serve
- This sequence is an algorithm

More Examples

➤ ATM Withdrawal

- Insert Card
- Enter PIN
- Select Amount
- Collect Cash

➤ Google Maps

- Source
- Destination
- Find shortest route
- Display route

➤ Cooking Recipe (every recipe is an algorithm)

Definition of Algorithm

Proper definition

- An **Algorithm** is a finite sequence of well-defined instructions that solves a problem or performs a task

Simple definition

- Algorithm = Step-by-step solution of a problem

Real example

- Find the largest number among three numbers

ALGORITHM: FINDING LARGEST NUMBER AMONG THREE NUMBERS

ALGORITHM (Step-by-Step)

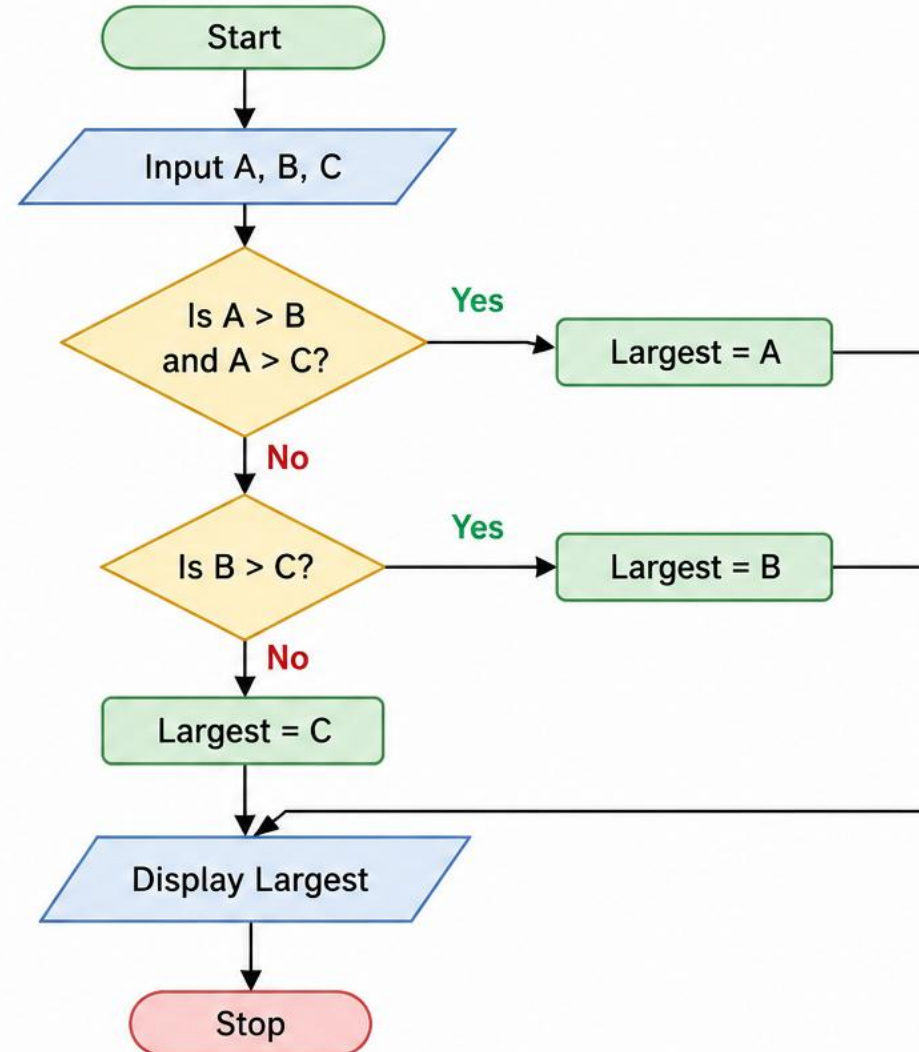
1. **Start**
2. **Input** three numbers A, B and C
3. **If** $A > B$ and $A > C$
 Largest = A
4. **Else if** $B > C$
 Largest = B
5. **Else**
 Largest = C
6. **Display** Largest
7. **Stop**

Example:

A = 25, B = 40, C = 35

Largest number is **40**

FLOWCHART



Characteristics of a Good Algorithm

1. Input
2. Output
3. Definiteness
4. Finiteness
5. Effectiveness
6. Correctness

Prepared By: Ehtisham Rasheed

Algorithm vs Program

Algorithm	Program
Logical solution	Actual implementation
Language independent	Language dependent
Easy to understand	Requires compiler/interpreter
Can be written on paper	Written in C++, Java, Python etc.

Abstract Data Type (ADT)

- An Abstract Data Type (ADT) defines **what operations** can be performed on data without specifying **how** those operations are implemented
- Remember:
- ADT = WHAT
- Data Structure = HOW

Real Life Example

- TV remote
- Operations:
 - Power
 - Volume Up
 - Volume Down
 - Channel
- We know what it does
- We don't know how the electronics work
- This is abstraction

Different ADTs

List ADT

- Operations (insert, delete, search, update, traverse)
- Implementation (can be implemented using array or linked list)

Stack ADT

- Operations (push, pop, top, isEmpty, size)
- Implementation (can be implemented using array or linked list)

Queue ADT

- Operations (enqueue, dequeue, front, rear, isEmpty)
- Implementation (can be implemented using array or linked list)

ADT vs Data Structure

Abstract Data Type	Data Structure
Defines WHAT	Defines HOW
Logical concept	Physical implementation
Operations	Memory organization
Language independent	Implemented in C++
Example: Stack, Queue, List	Example: Array, Linked List